

Year Up United

Data Analyst Training Academy:

Continuing with Python

Week 6 Lab Workbook

This courseware is to be distributed to accompany instructor led training.

Table of Contents

MODULE 1 JUPYTER NOTEBOOK	3
EXERCISE 1.A WORKING WITH JUPYTER NOTEBOOK.....	4
MODULE 2 EXPLORING FUNCTIONS	5
EXERCISE 2.A USING BUILT-IN FUNCTIONS	6
EXERCISE 2.B USING LIBRARY FUNCTIONS.....	11
EXERCISE 2.C CREATING AND CALLING FUNCTIONS.....	16
EXERCISE 2.D LAMBDA AND GENERATORS.....	20
EXERCISE 2.E USING FUNCTIONS FOR ANALYZING DATA.....	23
MODULE 3 CLASSES/OBJECTS	29
EXERCISE 3.A WORKING WITH CLASSES.....	30
EXERCISE 3.B MORE FUN WITH CLASSES.....	33
MODULE 4 FILES & EXCEPTIONS	36
EXERCISE 4.A PRACTICING READ/WRITE.....	37
EXERCISE 4.B WORKING WITH EXCEPTIONS.....	44
MODULE 5 NUMPY	46
EXERCISE 5.A NUMPY FUNDAMENTALS	47

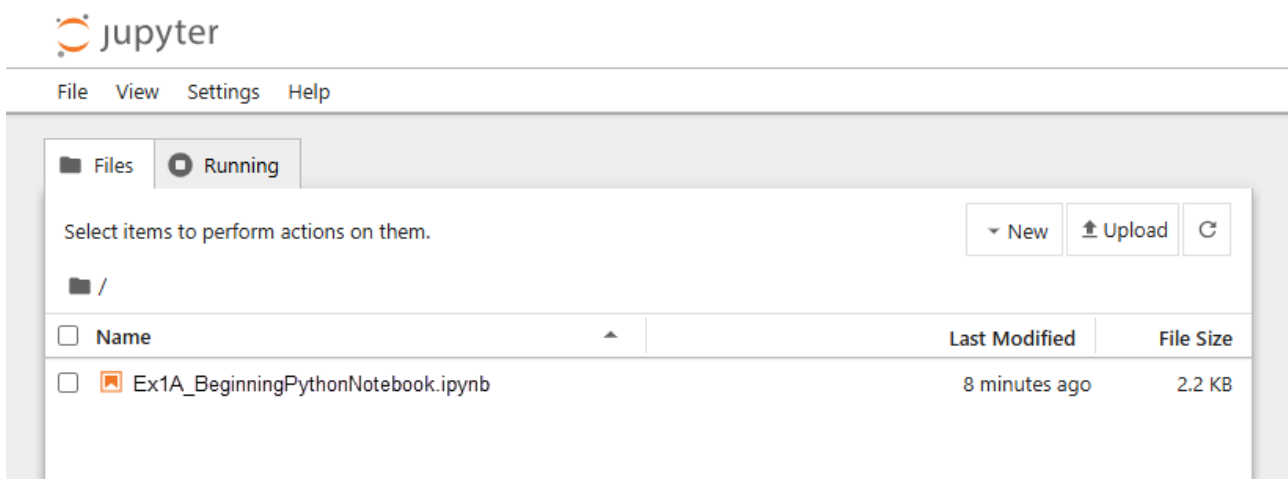
Module 1

Jupyter Notebook

Exercise 1.A Working with Jupyter Notebook

Starting with a Sample Notebook

1. From the Week 6 course material, download the notebook file **Ex1A_BeginningPythonNotebook.ipynb** and add it to your **DataAnalytics/week-06** folder.
2. Use Git Bash to navigate to **DataAnalytics/week-06** then type the command `jupyter notebook` to launch Jupyter Notebook.
3. In the browser window for Jupyter Notebook, open the sample notebook.



4. Work through the exercises in the notebook, which act as a refresher on the Python concepts you learned last week, while also getting you familiar with the layout of Jupyter Notebooks.
5. When you are finished, save your changes, then **shut down** Jupyter Notebook using the File menu.
6. When Jupyter Notebook shuts down, you will see the connection close in Git Bash. Your notebook changes are saved locally...
7. ... but you still need to sync them with your GitHub repository to submit the homework assignment! Use Git Bash to add, commit, and push changes for the week-06 folder.

Module 2

Exploring Functions

Exercise 2.A Using built-in functions

Lab 1

In DataAnalytics/week-06, add a new subfolder named **Ex2A_Functions**. The following exercises should be placed there.

1. Use Git Bash to navigate to DataAnalytics/week-06/Ex2A_Functions then use the command `jupyter notebook` to launch Jupyter Notebook.
2. Create a new notebook, using the File menu to choose New > Notebook.
3. At the top of the new notebook, double-click on the word Untitled and change the notebook name to **substrings.ipynb**.
4. At the top of your notebook, add a **markdown** cell with a description of this lab:

```
# Exercise 2.A Using built-in functions
```

```
## Lab 1. Working with names - parsing strings
```

You can see in the toolbar whether the selected/active cell is a “code” cell or “markdown” (or other type). Run the cell (shift+enter or run icon in the toolbar) to render the markdown. If your cell just renders as Python code comments (with the hashtags still visible, not as formatted headers), check in the toolbar that you have changed the cell type to markdown and then try running the cell again.

5. The next cell should be a code cell. Add a variable containing a full name, such as:

```
test_name = 'John Doe'
```

Run the cell (shift+enter or the run icon in the toolbar) to run code and commit the variable to memory. Once you run the cell, that variable will stay in memory until either you change it in another cell or you restart the kernel.

6. In the next cell, write a function named `parse_and_display` that takes one parameter, `name`. In the function, loop over the name and find the space, then

extract the first and last name based on string indexing. Then add a print statement to see the results. Your function might look something like this:

```
def parse_and_display(name):  
    for i in range(len(name)):  
        if name[i] == ' ':  
            first = name[:i]  
            last = name[i+1:]  
            print(f"Full name: {name}")  
            print(f"First name: {first}")  
            print(f>Last name: {last}")
```

7. Notice that when you run the cell with your new function, no output appears. That's because you actually need to call the function and supply an argument. In another code cell, call the function using the variable you previously created:

```
parse_and_display(test_name)
```

Run the cell to check the output, which should look like this:

```
Full name: John Doe  
First name: John  
Last name: Doe
```

8. Add another new code cell with a list of names to test:

```
test_list = ['Grace Flores', 'JB Oinonen', 'Ken Penn']
```

Again, make sure you run the cell to commit `test_list` to memory.

9. Use additional cells in the notebook to test your function with the names in the list. The full output (full name, first name, last name) should be printed for each name.

Example (do this for each list item in a separate cell):

```
parse_and_display(test_list[0])    # output for Grace Flores
```

10. Add a code cell testing your own name, inputting your name directly as a string:

```
parse_and_display('Your Name')
```

11. **CHALLENGE:** In a new cell at the bottom, create a modified version of your function called `parse_and_display2()` that will work with names made of one, two or three words. Output should include middle name if applicable.

Some more example names to use when testing your new function:

Lorde

Billie Eilish

Megan Thee Stallion

12. Save changes to your notebook when you are done. Then use Git Bash to commit and push changes to week-06.

Lab 2

1. In the folder Ex2A_Functions, launch Jupyter Notebook.
2. Create a new notebook named **part_codes.ipynb**.
3. At the top of your notebook, add a markdown cell with a simple description of this lab:

```
# Exercise 2.A Using built-in functions
```

```
## Lab 2. Parsing strings - part codes
```

Run the cell (shift+enter or run icon in the toolbar) to render the markdown

4. In this lab, you will create multiple functions that, together, can parse a part code which is provided in the following format:

```
supplier:productcode-size
```

Examples:

```
ACME:123-L    supplied by Acme, part #123, size large
```

```
DI:12345-M   supplied by DI, part #12345, size medium
```

```
ACE:1-12     supplied by Ace, part #1, size 12
```


5. In a separate code cell for each function, create the following:

```
get_supplier_code() → returns all characters before the :  
get_product_num() → returns all characters between the : and -  
get_size() → returns all characters after the -
```

Note: You are encouraged to ask your peer group for code review of each other's ideas. Also be sure to test each function as you go, and add comments to explain your code.

6. Once you have your functions ready, in another code cell create variables to hold the three example part codes from above (if you haven't already done so).
7. One at a time in new code cells for each test example, call all three functions for each part code and use print statements to output the results as follows:

For [full product code], the supplier is [supplier]. Part number [part #] is available in size [size].

8. BONUS (optional): Try creating a variable that contains all three part codes as a list. How can you get your script to work with this list input?
9. Remember to save your notebook, and use Git Bash to commit and push changes to week-06 when you are done!

Lab 3

1. In the folder Ex2A_Functions, make a new notebook named `update_hr_doc.ipynb`.
2. Add a markdown cell to the top of the notebook with a description of this lab:

```
## Lab 3. Employee data  
This lab exercise makes updates to information extracted from  
an employee database.
```

As always, run the cell to render the markdown.

3. Add a code cell and define the variable `hr_list` to include the below list of tuples:

```
hr_list = [  
    ('0123', 'HR', 'Rebecca Yang', '69000'),  
    ('0121', 'IT', 'Mark Blick', '67000'),  
    ('0068', 'IT', 'Kahna Larsen', '112000'),  
    ('0234', 'OPS', 'Jim Smith', '54000')  
]
```

4. Make your first update: Mark just reported a name change.

a) In one code cell, write a piece of code to update the list to change Mark's last name to Blick-Hawley.

b) In another code cell, print the updated list output.

5. Make your second update: Jim has accepted a new account manager role in another department.

a) In one code cell, write a piece of code to change Jim's department code from OPS to CS and update his salary to 60000.

b) In another code cell, print the updated list output. (Make sure both the changes to Mark's name and Jim's department persist.)

6. Add another code cell to display the updated contents on multiple lines in the following format:

a) Use the pipe/bar character (that's | or shift+\ on your keyboard) to separate each value, with a space on either side of the pipe character, and format the salary to use a comma:

Employee# | DeptCode | Name | NN,NNN

Note: Formatting that last value can be deceptively tricky! Consider that you might have to cast it as another data type when using the format function...

7. Remember to save your notebook, and use Git Bash to commit and push changes to week-06 when you are done!

Exercise 2.B Using library functions

Lab 1

In DataAnalytics/week-06, add a new subfolder named **Ex2B_Functions**. The following exercises should be placed there. Note that some of these exercises use VS Code, and some use Jupyter Notebook.

1. In the folder Ex2B_Functions, use VS Code to create a new script file named **sampling_tools.py**.
2. Begin your script by importing the random module at the top.
3. Add the following starting list (a product inventory a data analyst might work with). Then run the script to confirm there are no errors before continuing:

```
products = ['Laptop', 'Monitor', 'Keyboard', 'Mouse', 'Webcam',  
'Headset', 'Docking Station', 'USB Hub', 'Desk Lamp', 'Surge  
Protector']
```

4. For each of the following hypothetical scenarios, add the code to your script and run it to check the output before moving on to the next:
 - a) The team wants to feature a "Product of the Day" based on one randomly selected item. Use `random.choice()` to select one product and print the result with an appropriate label. Be sure to run the script a few times to confirm the selection changes with each run.
 - b) Three products need to be selected for a brief usability survey. The same product should not appear more than once. Use `random.sample()` to select 3 items from products without replacement and print the results.
 - c) For a presentation, all products should be displayed in a randomized order to avoid any appearance of ranking. Use `random.shuffle()` to shuffle the products list, then print the shuffled list.

Note: Unlike `.choice()` and `.sample()`, `random.shuffle()` does not return a shuffled list but rather modifies the list in place. Print `products` after shuffling, not the return value of `random.shuffle()`, or you will get an output of `None`.

- d) Use `random.randint()` to generate a simulated daily transaction count between 50 and 300, and print the result with a label.
5. When you are finished, run the complete script a few times and observe how the output changes each run.
6. Remember to save your script, and commit and sync changes when you are done!

Lab 2

1. Use Git Bash to navigate to `DataAnalytics/week-06/ Ex2B_Functions` and launch Jupyter Notebook.
2. Create a new notebook named `times_and_dates.ipynb`.
3. Add a markdown cell at the top to explain that this is Lab 2 of Exercise 2.B, working with datetime functions.
4. In the first cell, begin by importing the datetime module (`import datetime`).
5. Add a markdown cell with the following heading:

```
# Today
```

6. In a code cell, define a variable called `today` that represents the current date and time using `datetime.datetime.now()`
7. Add another code cell. Using the `.strftime()` method, identify the following pieces of information and print the output:

```
Today is [day of week], [month] [day], [year]
```

8. Add a markdown cell with the following heading:

```
# My birthday
```

9. In the next cell, define a variable for your birthday using `datetime.date()`.

10. In the next cell, again using the `.strftime()` method, create the following output:

```
My birthday is MM/DD/YYYY
```

11. Add a markdown cell with the following heading:

```
# Next 90 Days
```

12. In the next cell, define a variable for `ninety_d` which will be used to represent the date 90 days from today, and set the value of this variable to add the following values:

```
today + datetime.timedelta(days=90)
```

13. In the next cell, again using the `.strftime()` method as needed, print the following output:

```
90 days from today is [month] [day], [year]
```

14. Add a markdown cell with the following heading:

```
# Time for Dinner
```

15. Define a variable for `dinner_time` using `datetime.time()`.

16. Again using the `.strftime()` method, create the following output:

```
Let's meet for dinner at HH:MM PM
```

17. Remember to save changes to your workbook, then use Git Bash to commit and push changes to week-06 when you are done!

Lab 3

1. Using VS Code, in the folder `Ex2B_Functions`, make a new file named `math_and_stats.py`. Begin by importing three modules:

```
import random
import math
import statistics
```

2. Create a few starting variables to work with:

```
vals_1_100 = range(1,100)
vals_sample = random.sample(vals_1_100, 75)
vals_choices = random.choices(vals_1_100, k = 200)
radius = random.randint(3,10)
pi = math.pi
```

3. Use a combination of functions from all three modules to create calculations that will support the following output (and be sure to use comments to document your code as you work!):

_Experimenting with a subset of integers 1-100:

Sum of 75 sample values from 1 to 100: ____

Average of 75 sample values: ____

Median of 75 sample values: ____

_Experimenting with a superset of 200 values, integers 1-100:

Average of 200 values: ____

Median of 200 values: ____

Mode of 200 values: ____

Standard deviation of 200 values: ____

Variance of 200 values: ____

_Modeling a random circle:

Radius = __, area = ____ (rounded up to the nearest integer)

Radius = __, area = ____ (rounded down to the nearest integer)

a) Your final print statement should include the printed headers each beginning with an underscore and the line breaks between sections in your output. For the line breaks, use `print('\n')`

b) Your calculated answers should fill in the blanks above.

- c) While some starting variables have been provided at Step 2 above, you will likely want to create additional variables to use in each step of your calculations. Don't try to do the whole calculation in the print line!!
- d) If you need a reminder of how to calculate the area of a circle, in math the formula is usually written as:

$$area = \pi(r^2)$$

where π is the symbol for pi and r represents radius, which must be squared. You'll need to translate this into a Python expression for *pi times radius squared*.

- 4. Remember to save your script, and commit and sync changes when you are done!

Exercise 2.C Creating and calling functions

Lab 1

In DataAnalytics/week-06, add a new subfolder named **Ex2C_Functions**. The following exercises should be placed there.

1. Use Git Bash to launch Jupyter Notebook from week-06/ Ex2C_Functions, then create a new notebook named **simple_functions.ipynb**.
2. Add a markdown cell at the top to explain that this is Lab 1 of Exercise 2.C, working with defining custom functions.
3. In a code cell, create a function named `favorite_things()` and include three parameters for `name`, `movie`, and `music`.
 - a) This function should return 3 lines of text listing your name, your favorite movie, and your favorite musician / band.
 - b) In a print statement, call the function with appropriate arguments to display the output.
4. In another code cell, create a function named `why_im_here()` with one parameter called `purpose`.
 - a) This function should return the value supplied to `purpose`, which will be a sentence that describes why you joined the Year Up United Data Analyst program.
 - b) In a print statement, call the function with appropriate arguments to display the output.
5. In another code cell, create a function named `favorite_place()`, with two parameters, `place` and `why`.
 - a) With this function, you will output a sentence that will describe one of your favorite places to visit and why, e.g. *One of my favorite places is ____ because ____.*
 - b) Add two code cells to test. In one, call the function with one of your favorite places and display the output.

- c) Then in the next code cell, provide a different place name and reason why it's one of your favorites.
6. When you are done, review your code with your peer group. How did each of you approach these functions? Which ended up seeming to be the most efficient? Which would be the most readable / reusable to another coder if the function were shared?
7. Remember to save your notebook, and commit and push changes to week-06!

Lab 2

1. Using VS Code, add a new script to the Ex2C_Functions folder named **more_functions.py**. In it, define three functions as follows.
2. The first should be named `display_mailing_label()`, with five parameters: `name`, `address`, `city`, `state` and `zip`. In the function output, format and display the data as you would on an address label.
3. The second function should be named `add_numbers()` with one parameter defined to accept any number of arguments, each argument being an integer. In the function, add given arguments together and display the result using the following format:

number [+ number2 + number3 ...] = result

4. The third function should be named `display_receipt()` and accept two parameters: `total_due` and `amount_paid`. Compute and display the change due in the following format:

Total Due: \$_____

Amount Paid: \$_____

Change Due: \$_____

If the amount paid is less than the total due, display a message indicating the remaining balance to be paid.

5. When you have defined each function:

- a) Call `display_mailing_label()` at least twice with data for two different people.
 - b) Call `add_numbers()` three times with one number, two numbers, and your choice of however many numbers (more than two).
 - c) Call `display_receipt()` three times. The first time, overpay the bill. The second time, pay the bill exactly. The third time, underpay the bill.
6. BONUS (optional):
- a) Some addresses may use two lines, like a business name plus street address, or an address plus a suite/apartment/unit number, or a “care of” (c/o) line plus address. Add a modified version of the `display_mailing_label()` function to allow for an address to include one or two lines.
 - b) What if the bill-payer has multiple balances to pay? Add a modified version of the `display_receipt()` function to allow for one or more values to be input for the total due. The output of the function should remain the same.
7. Remember to save, commit, and sync changes when you are done!

Lab 3

1. From the week-06/ Ex2C_Functions folder, use Git Bash to launch Jupyter Notebook and create a new notebook named `tax_functions.ipynb`.
 - a) OPTIONAL: Alternatively, use VS Code to create the new file `tax_functions.ipynb` in your week-06/ Ex2C_Functions folder. Then proceed with working with the rest of this lab using VS Code.
2. Use separate code cells to create three functions:
 - a) `get_soc_sec_tax()` accepts a gross pay amount and returns the Social Security tax on that amount. Assume a tax rate of 6.2%
 - b) `get_medicare_tax()` accepts a gross pay amount and returns the Medicare tax on that amount. Assume a tax rate of 1.45%
 - c) `get_federal_tax()` accepts a gross pay amount and withholding code, and returns the federal tax withholding on that amount. Use the tax table below. (You will need to use an `if / else` statement. The case for withholding code 4+ is extra

tricky!)

Withholding Code	Tax Rate
0	23%
1	21%
2	19.5%
3	18.5%
4+	18% less 0.5% for each withholding over 4 (ex, code 6: rate $.18 - (2 \times .005) = .17$ (or 17%))

3. Create three more code cells to test your functions, one cell for each of the following examples:

Person1: gross pay \$750 withholding code 0
Person2: gross pay \$1550 withholding code 2
Person3: gross pay \$1100 withholding code 5
4. Go back to the top of your notebook and insert a markdown cell with a description of this lab exercise and what it demonstrates.
5. Remember to save changes to your notebook, then use Git Bash to commit and push your changes to week-06.

Exercise 2.D Lambdas and generators

Lab 1

In DataAnalytics/week-06, add a new subfolder named **Ex2D_Lambdas&Generators**. The following exercises should be placed there.

1. Using VS Code, create a new file named **quick_multiplier.py**.
2. Start by creating a variable named `doubler` that uses a lambda function to double whatever argument it receives, like this: `doubler = lambda n: n * 2`
3. Print the variable multiple times to test it out with the following values:

8
-4
'banana'
4. Create a `tripler` variable using similar logic but multiplying the supplied argument by 3 (instead of 2), and test it out with the same sample values.
5. If you want to create a similar multiplier variable for numbers 4 through 10, how can you save yourself some code by putting this lambda within a function? Create a `multiplier()` function and use it to create the following variables:

`quadrupler` (or you could name your variables *times4*, *times5*, etc.)
`quintupler`
`sextupler`
`septupler`
`octupler`
`nonupler`
`decupler`

Hint: Refer back to the example in 2.D.2 for reference.

6. Print each of the new variables at least once, with a sample value as the argument.

7. Remember to commit and sync changes when you are done.

Lab 2 – OPTIONAL

This is a fun lab if you have time to tackle it:

1. Are you familiar with the song “The Name Game”? If not (or if you’d like a refresher), check out this recording of Shirley Ellis performing the original version:
https://www.youtube.com/watch?v=5MJLi5_dyn0
2. Using VS Code, create a new file named `name_game_bonus.py`.
3. First, define an input variable to prompt the user for a name.
4. Next, create a function called `trunc_name` to create a truncated version of the name for using in the song.
 - a) Consider how to account for inputs that might use different capitalization. The function output should always be in lower case.
 - b) You will need to implement some `if/elif/else` logic to account for differences in names beginning with a vowel, one consonant, or two consonants. For instance, here is what the truncated output should be for each of the following examples:

Ann : ann

Dan : an

Stan : an

5. Print the output to test your function with different name inputs. When you are satisfied with your `trunc_name` function, comment out the print line.
6. Define another function called `name_game()`. This will be a generator function.
 - a) Create a separate `yield` statement for each line of the name game, e.g.:

Anna, Anna, bo-banna

banana fana fo-fanna

me my mo-manna

Anna!

7. Using a `for` loop, print the output of the `name_game()` function for these examples:

[Your own name]

carly

CHARLIE

Aidan

Braden

Billy Bob

Run into any error messages? Unexpected outputs? Correct your code logic for any error messages, and add a comment line at the bottom for other observations.

8. CHALLENGE: The Wikipedia page for the song “The Name Game” (https://en.wikipedia.org/wiki/The_Name_Game) includes an explanation of the rules, including a special rule for names that start with *b*, *f*, or *m*. How could you update your script to account for this special rule?
9. CHALLENGE: There are a few names that are famously considered naughty or inappropriate for the family-friendly Name Game, such as Bart and Buck. How would you choose to handle these inputs to your script? You might write a warning message to the user and re-prompt for the name again to confirm, or you might update the output for these specific cases. Pick the approach that you prefer and implement it in your script.

Remember to commit and sync your changes when you finish the labs.

Exercise 2.E Using functions for analyzing data

Setup

1. In DataAnalytics/week-06, use either Jupyter Notebook or VS Code to create a new notebook named **Ex2E_sales_records.ipynb**
2. Add a markdown cell at the top of the notebook with the following content (copy then update as needed):

```
# Sales Records Analysis
```

```
### Author: [Your Name]
```

```
### Exercise 2.E Using functions for analyzing data
```

```
This notebook analyzes a small sales dataset using Python functions.  
Each function takes the full dataset as its first argument.
```

When run, the text should look something like this:

Sales Records Analysis

Author: Bilbo Baggins

Exercise 2.E Using functions for analyzing data

This notebook analyzes a small sales dataset using Python functions. Each function takes the full dataset as its

3. Add a code cell creating the variable `sales_data` with the below values, which you will use throughout the exercise. You'll notice that each list item in `sales_data` is a tuple, representing a record that contains a product name, sales region, and sale amount. Run the cell before continuing.

```
sales_data = [  
    ('Laptop',      'East',   1200.00),  
    ('Phone',       'West',   850.00),  
    ('Tablet',      'East',   400.00),  
    ('Monitor',     'South',  650.00),  
    ('Keyboard',    'West',   75.00),  
    ('Laptop',      'South',  1150.00),  
    ('Phone',       'East',   900.00),
```

```
( 'Tablet',      'West',      425.00),
( 'Monitor',     'East',      680.00),
( 'Keyboard',    'South',     80.00),
( 'Laptop',      'West',     1300.00),
( 'Phone',       'West',      820.00),
]
```

Lab 1. Total sales by region

4. Add a markdown cell with the name of the lab as a subheader:

```
## Lab 1. Total sales by region
```

5. Add a code cell and write a function named `region_total(data, region)` that:

- Takes the full dataset and a region name as arguments
- Returns the total of all sale amounts for that region

Hint: generator expressions

A generator expression lets you loop and filter in a single line inside a function call like `sum()`. as covered in Section 2.D of the Week 6 Student Guide.

For example, to sum only the sale amounts where the region matches, you can use:

```
def region_total(data, region):
    return sum(amt for _, r, amt in data if r == region)
```

Notice the underscore `_` in the sequence `_, r, amt`. This is a Python convention for a variable you are unpacking but do not need to use.

Copy the function above into your code cell. Then run the cell.

6. Add a code cell, and write three print statements to test the function:

```
print(region_total(sales_data, 'East'))
print(region_total(sales_data, 'West'))
print(region_total(sales_data, 'South'))
```

Run the cell and confirm the output before continuing.

Lab 2. Count of sales by region

7. Add a markdown cell with the name of the lab as a subheader:

```
## Lab 2. Count of sales by region
```

8. Add a code cell and write a function named `region_count(data, region)` that:
- Takes the full dataset and a region name as arguments
 - Returns the **number of records** for that region

Hint: You can use the same generator expression pattern from Lab 1. Instead of summing `amt`, simply sum `1` for each matching record.

9. Add a code cell, and write three print statements to test your function with all three regions.

Lab 3. Average sale amount by region

10. Add a markdown cell with the name of the lab as a subheader, following the same pattern as you did for Lab 1 and Lab 2.
11. Add a code cell and write a function named `region_average(data, region)` that:
- Takes the full dataset and a region name as arguments
 - Returns the average sale amount for that region
 - Returns 0 if the region has no records (to avoid a division by zero error)

Hint: You already have `region_total()` and `region_count()`. For this lab, call them from inside `region_average()` rather than re-writing the logic.

12. Add a code cell, and write three print statements to test your function with all three regions. Also add a print statement to test the output when you use the region 'North' (which should return zero records from `sales_data`).

Lab 4. Filter by sale amount

13. Add a markdown cell with the name of the lab as a subheader, continuing to follow the same pattern.
14. Add a code cell and write a function named `sales_above(data, threshold)` that:
 - a) Takes the full dataset and a numeric threshold as arguments
 - b) Returns a new list containing only the records where the sale amount is greater than the threshold
15. In the next cell, test your function using a threshold of 500. Print each record in the returned list on its own line using a `for` loop.
16. **Bonus:** In another code cell, write a `sales_below(data, threshold)` function using the same pattern. Test the results for sales of less than 500.

Lab 5. Top and bottom sale

17. Add a markdown cell with the name of the lab as a subheader, continuing to follow the same pattern as above.
18. Add a code cell and write two functions:

`top_sale(data)` - returns the single record (tuple) with the highest sale amount

`bottom_sale(data)` - returns the single record (tuple) with the lowest sale amount

Hint: The built-in `max()` and `min()` functions accept an optional `key=` argument that tells them how to compare the items in a collection. Use it to pass a function (often a lambda) that extracts the comparison value from each item.

For example, to find the tuple with the highest sale amount (at index 2):

```
max(data, key=lambda s: s[2])
```

19. After writing and testing both functions, add a markdown cell describing the result in plain English. For example: "The top sale was a [product] in the [region] region for \$[amount]."

Lab 6. Apply a discount

20. Add a markdown cell with the name of the lab.
21. Add a code cell and write a function named `apply_discount(data, discount_pct)` that:
- a) Takes the full dataset and a discount percentage (e.g., ``0.10`` for 10%) as arguments
 - b) Returns a ****new list**** of records with the sale amount reduced by the discount percentage
 - c) The product name and region in each record should stay the same; only the amount changes

Hint: `map()` applies a function to every item in a collection. You can use a lambda as that function. To return a new tuple with the amount updated:

```
lambda s: (s[0], s[1], s[2] * (1 - discount_pct))
```

Wrap the `map()` call in `list()` to get a plain list back.

22. Test your function with a 15% discount. Print the first three records before and after the discount to confirm the amounts changed correctly.

Lab 7. Unique regions

23. Add a markdown cell with the name of the lab.
24. Add a code cell and write a function named `unique_regions(data)` that:
- a) Takes the full dataset as its argument
 - b) Returns a sorted list of unique region names in the dataset

Hint: Use a generator expression inside `set()` to extract the region from each record and automatically deduplicate, then wrap in `sorted()`.

25. After testing, add a markdown cell explaining what this function could be useful for if you were writing a longer analysis program. What kind of information or analysis can you use this pattern for? (There is not a definitive right or wrong answer, just give it your best thoughtful response.)

Lab 8. Region summary

26. Add a markdown cell with the name of the lab.
27. Add a code cell and write a function named `region_summary(data, region)` that:
- a) Takes the full dataset and a region name as arguments
 - b) Pulls together summary information by calling `region_total()`, `region_count()`, `region_average()`, and `top_sale()` (using a list comprehension to narrow to the specific region first)
 - c) Prints a formatted report for that region, for example:

```
=== East Region ===  
Records:    4  
Total Sales: $3,180.00  
Average Sale: $795.00  
Top Sale:    Laptop — $1,200.00
```

Note: You may need to think carefully about how to find the top sale for a specific region using functions you have already written. You have at least two options, so use whichever you prefer.

Lab 9. Full summary

28. Add a markdown cell with the name of the lab.
29. Add a code cell and write a loop that calls `region_summary()` for every region in the dataset without hardcoding the region names.

Hint: You already wrote `unique_regions()` so how can you use that here?

30. Add a final markdown cell with a summary of what you learned from this set of labs.

Module 3

Classes/Objects

Exercise 3.A Working with classes

Lab 1

In DataAnalytics/week-06, create a new subfolder named **Ex3A_Classes**. The following exercises should be placed there.

1. Using VS Code, create a new script named **restaurants.py**.
2. Make a class called `Restaurant`. The `__init__()` method for `Restaurant` should store two instance variables: `rest_name` and `food_type`. Be sure to include a `"""docstring"""` below the class name to describe the purpose of the class.
3. Add two methods to the class:
 - a) The first method should be named `describe_rest()` and prints the output: `[Restaurant name] serves [type of food]`.
 - b) The second method should be named `rest_open()` and prints a simple message: `[Restaurant name] is open`.
4. Create three instances of the class for different types of restaurant. You can use the below examples or create your own:



5. Finally, call `describe_rest()` and `rest_open()` for each instance.
6. Remember to commit changes when you are done.

Lab 2 - OPTIONAL

1. Create a new script named `rewards_prog_bonus.py`.
2. Make a class called `RewardsProgram`. Be sure to include a docstring below the class name to describe the purpose of the class. The `__init__()` method for `RewardsProgram` should store the following instance variables:
 - a) `cust_name` for customer name
 - b) `phone` for customer phone
 - c) `email` for customer email
3. Add two methods to the class:
 - a) The first method should be named `profile()` and prints the output:

Name: [customer name]
Phone: [customer phone]
Email: [customer email]
 - b) The second method should be named `thank_you()` and prints the message:
Thank you, [name], for visiting our restaurant!
 - c) The third method should be named `add_to_cust_list()`. This should add the customer's name, phone, and email as a tuple to a list named `cust_list`, which should be global in scope. (Where should this variable be created to ensure that running this method adds to it and doesn't overwrite any existing values?)
4. Create three instances of the class, making up sample data for three different customers. For each customer, run the `profile()`, `thank_you()`, and `add_to_cust_list()` methods.
5. Finally, print the contents of `cust_list` to confirm that all customers have been added.

- a) If only one customer appears in the list, review your `add_to_cust_list()` method to ensure that you are appending new customer data, not overwriting it. Also check that your `cust_list` variable is only created once when your script runs, not recreated for each instance.

6. Remember to commit and sync changes when you are done.

Exercise 3.B More fun with classes

Lab 1

In DataAnalytics/week-06, create a new subfolder named **Ex3B_More_Classes**. The following exercises should be placed there.

1. Create a copy of your `restaurants.py` script from the last set of labs, save it to `Ex3B_More_Classes`, and rename it **`restaurants_enhanced.py`**.
2. Add a default attribute to the `__init__()` method for `self.number_served = 0` and another default attribute for `self.customer_ratings = []`
3. Add a method called `add_num_served()` that accepts an input for “How many customers served today?” and adds that amount to the `self.number_served` attribute.
4. Add a method called `print_num_served()` that prints the output:
[Restaurant name] has served [number] customers
5. Add a method for `customer_rating()` that accepts an input of integers 1-5 for “How would you rate your experience today on a scale of 1-5 (5 being excellent)?” Add that number to the list of `self.customer_ratings`, and print the statement “Your rating was _____. The average rating for this restaurant is _____” (You can calculate the average using the sum of the ratings list, divided by the list length.)
6. Test your new methods:
 - a) For each of your example restaurants, run `print_num_served()` to check the initial value. Then run `add_num_served()` a few times, inputting different values. Finally, run `print_num_served()` again to check the updated balance.
 - b) For each of your example restaurants, run `customer_rating()` several times, inputting a different rating each time. Confirm that the average rating updates appropriately each time.
 - c) For `customer_rating()`, try inputting a few “incorrect” values, like the number 6, a decimal number such as 2.5, and a word/phrase such as “5 stars!”. Does your

script return a result? An error? Update the method with conditional logic to account for different possible input types, so that if something other than an integer 1-5 is entered, the user is prompted to reenter their rating.

7. Remember to commit changes when you are done.

Lab 2 – OPTIONAL (prerequisite: Ex 3.A Lab 2)

1. Create a copy of your `restaurants.py` script, save it to `Ex3B_More_Classes`, and rename it as `rewards_prog_bonus_enhanced.py`.
2. Add a default attribute to the `__init__()` method for `self.restaurants_visited = []` and another default attribute for `self.rewards_points = 0`
3. Add a new method called `visit_rest()` that accepts two inputs:
 - a) Prompt for “Name of restaurant: ”. Check if the restaurant is currently in `self.restaurants_visited`, and if not, add it to the list.
 - b) Prompt for “What was the total food bill for this visit?” and convert that to a rewards point amount at a rate of \$1 to 1 point. Rewards points should round down to the nearest whole number. Add the points earned to `self.rewards_points`. Print the output:

Points for this visit: ____
Total rewards points earned: ____
Thank you for visiting [name of restaurant]!
4. CHALLENGE: Right now, your script tracks a single balance for total rewards points. Update your script to store rewards points as a dictionary, separately tracking the points balance for each restaurant visited.
5. CHALLENGE: At this point, your `visit_rest()` method is doing quite a lot! Create a new method for `calculate_rewards()` and move the points calculation logic from `visit_rest()` to the new method, then have `visit_rest()` call that method to calculate and supply the rewards points.

Lab 3 - OPTIONAL

1. Create a copy of your `restaurants.py` script saved to `Ex3B_More_Classes` and rename it `food_truck_bonus.py`.
2. At the bottom of the script following the `Restaurant` class creation and object initialization, define a new child class named `FoodTruck`. Remember to include the name of the parent class in parentheses in the class definition, and include a docstring to describe the child class.
3. Define the `__init__()` method to take in the information required in the parent `Restaurant` class, and use the `super()` function to call the `__init__` method from the parent class.
4. Add two new attributes to the `__init__()` method specific to the child class:
 - a) `self.private_bookings` with a default value of `'N'`
 - b) `self.truck_location` with a default value of an empty string
5. Add a method called `accepts_private_bookings()` that prompts for an input with “Does this food truck accept private bookings? Y/N ” and updates the value of `self.private_bookings`. Then print the appropriate message: “This food truck currently accepts [or does not accept] private bookings.”
6. Add a method called `relocate_truck()` that prompts the user to enter the truck’s current location (street address and city). Print the output, “Truck is currently located at [address, city]”
7. CHALLENGE: How would you add a history of the locations that the truck has used? Decide whether to include duplicate locations or just one entry per location. (Why might you choose either approach? What other information might be relevant? Include a note in comments to briefly explain your chosen approach.) Test your implementation by printing the location history.

Remember to commit and sync changes when you are done.

Module 4

Files & Exceptions

Exercise 4.A Practicing read/write

Lab 1

Create subfolder in DataAnalytics/week-06 named **Ex4A_ReadWriteFiles** and save the following exercises there

1. Create a new script named **open_and_write.py**.
2. Begin the script by creating a new variable to work with a file using the open function and including the append parameter, and create a new file named `about_me.txt`. Then close the file.
 - a) Need a hint? For the variable, you can use the convention of naming it `f`. Refer to 4.A.2 and 4.A.4 for reference on the rest of the syntax to use.
3. Run the script to create and close the empty file.
4. Using Visual Studio Code (or Notepad if you prefer), open the new **about_me.txt** and add the following information:

(Note: If you prefer not to share information about yourself OR if you get a kick out of creative writing, feel free to make up a profile of a fictional person)

- a) Name:
 - b) Place of birth:
[be as general or specific as you want, e.g. “Tennessee” or “I was born in Australia, then my parents moved to the US when I was 3” are both good answers]
 - c) Did you have any pets growing up? What kind? What were their names?
 - d) If you could travel anywhere in the world for ONE WEEK, where would you go?
 - e) If you could live anywhere in the world for a YEAR, where would you want to live?
5. Close **about_me.txt** and return to **open_and_write.py** in Visual Studio Code.

6. Update your script to add a new line, after opening the file, to use the `.write()` method to add the following additional information to the `about_me.txt` file:
 - a) If you could do anything for your "perfect" night out, where would you go and what would you do?
7. Run the script to update and close the file.
8. Open and review `about_me.txt` (using VS Code or Notepad). Were your additions appended as you expected?
 - a) If not, delete the new lines, save the file, and update and re-try your code.
 - b) Tips:
 - (1) You can use `.write()` multiple times to add multiple lines – everything doesn't have to fit into one `.write()` statement.
 - (2) Remember that you can add line breaks in a string two ways, either using `\n` or using enter within a triple-quote string (`"""`). Note that using the latter method, any indentation will also be captured by the string, so don't indent text if you don't want it indented!
9. Save changes to your script.
10. Now, create a new script named `open_and_read.py` that includes the following steps:
 - a) Open the file `about_me.txt` in read mode.
 - b) Print the contents of the file to the terminal using `.read()`.
 - c) Close the file.
11. Run your script and review the results to confirm the script works as expected.
12. Change the print statement to use `.read(50)`, and add a second print statement also using `.read(50)`. How does this change the output? What's happening here?
13. Now comment out the print lines using `.read()`, and try experimenting with the `.readline()` method:

- a) Add a first print statement using `.readline(10)` and a second print statement using `.readline()` (no argument specified)
 - b) Add a simple for loop using the following format (if you are using a variable name other than `f`, make sure to update the code in the loop accordingly):

```
for i in range(1, 5):  
    print(f.readline())
```
 - c) Experiment with running all the `.readline()` statements together versus commenting out one or more statements before the loop, and observe the effect on the output.
14. Now comment out the line using `.readline()`, and try experimenting with the **`.readlines()`** method:
- a) What do you get when you print using `.readlines(1)` with no argument specified?
 - b) Add another print statement using `.readlines(1)` – what do you get?
 - c) Add a print statement using `.readlines(10)` – what do you get?
 - d) Add another print statement using `.readlines(10)`, and comment out the first two print statements using `.readlines(1)` – now what do you get?
 - e) What do you get with `.readlines(100)`? How about `.readlines(-1)`?
15. Let's try combining different types of read methods. First comment out all the print statements in your script. Then add three new variables with the following:
- a) One variable using `.read(50)`
 - b) A variable to capture the output of a loop using `.readline()` – you'll need to update the loop logic to append to the new list rather than print
 - c) A variable using `.readlines(100)`
16. Using the three new variables, print the following output:
- First 50 characters: [first variable]

Next four lines, as list by line: [second variable]

Next 100 characters, as list by line, rounded up to complete lines: [third variable]

17. Save changes to your new file, and remember to commit all changes when you are done.

Lab 2 – OPTIONAL

This is a fun lab if you have time to tackle it!

1. Is your birthday contained in pi? Start by locating the file `pi_million_digits_bonus.txt` in the student material, and download it to your `Ex4A_ReadWriteFiles` folder.
2. Create a new notebook named `pi_birthday.ipynb`. (You may use either VS Code or Jupyter Notebook for this.)
3. Start by creating a markdown cell with a description of this lab.
4. In a code cell, write code to open `pi_million_digits_bonus.txt` in read mode using the variable name `pi`, print the first line of characters, then close the file. Run the cell to confirm the output.
5. Duplicate your last code cell (or copy and paste the script from Step 4 into a new code cell). Delete the print statement, and in its place define a new variable named `pi_lines`. Assign to it the value `pi.readlines()` so that `pi_lines` contains a list of all the lines of the text file.
 - a) CAUTION: Printing the full contents of `pi_lines` won't break your computer, but still is not recommended! To test the variable, change to `.readlines(500)` to limit the size of the output. Then change it back to `.readlines()` when you are done testing.

(If you reeeally want to print the full contents and see what a million characters looks like in the terminal, no one's stopping you. But it's not necessary for this exercise. Remember that the interrupt command for the terminal is Ctrl+C)

6. Once you've used the `.readlines()` method to get the file contents into a list object, you're just about ready to iterate over that object using a `for` loop. But first you need to create the variable to hold the value that you'll be searching for.

- a) In a new code cell, create a function named `pi_bday_check(bday)` that incorporates a loop with the following logic:

For `i` in `pi_lines`, check if `user_bday` is in `i`. If so, print "Your birthday is in the first million digits of pi" and break. If not, continue. (Do *not* include an else block at this point.)

7. In the next cell, test the function by calling with a six-digit birthday *as a string*, e.g. '070281'.

- a) **Reminder:** if something goes sideways with your loop and you find yourself stuck in an infinite loop (or printing a response for every single line of a verry long list), hit **Ctrl+C** to interrupt the program.
- b) Want to test on some sample birthdays that you can be sure will work? Try:

July 2 1981
Oct 07 1999
Aug 15 2005

And a few that are *not* in pi:

Nov 2 1952
Nov 29 2001
Jan 24 2011

8. You may have noticed when testing your function that for birthdays that don't work, right now the script just ends – no error message but also no notification of the results. Make this a better experience when a birthday is not found.

- a) In a new code cell, copy in the code defining `pi_bday_check()` and change the function name to `pi_bday_check2()`, then make the following modifications:
- b) You can't just add a notification message to the loop's `else` block. If you try, you'll find that it prints for every single not-found line, which is pretty huge!

- c) Instead, you'll need a way to track whether or not the birthday was found, then deliver a message after the loop is done. You can do this by creating an initial empty variable *before* the loop to use as a tracker. Name the variable `birthday_found` and set its initial value to 0.
 - d) Update the loop's `if` block to include a new line *after* the print statement, with `birthday_found += 1`
 - e) Below the loop block, add a new `if` block to check if the value of `birthday_found` is *not equal* (`!=`) to 1, and if so, print the statement "Sorry, your birthday was not found in the first million digits of pi"
 - f) Be sure to test `pi_bday_check2()` with examples that are found and are not found, using the examples provided above.
9. Ok, you have confirmation as to whether a given birthday appears in the first million digits of pi. But *where* does the birthday actually appear within pi? At how many digits?
- a) To get a better understanding of the format of each line, use a new code cell to print the first two list items of `pi_lines` using their index number, and also print the length of each using `len()`, like so:

```
print(pi_lines[0])
print(pi_lines[1])
print(len(pi_lines[0]))
print(len(pi_lines[1]))
```
 - b) Your output should show that the first line starts with 3 . and is followed by a line break, while the second line starts with two spaces and is followed by a line break, and that the length of both lines is 103 characters.
10. In order to find the actual position of the birthday within pi, you'll need to concatenate the lines from `pi_lines` into one long string, but *without* the extra spaces or newline characters.
- a) In a new code block, create a variable `pi_string` and assign it a blank string (`' '`). Then create a new loop that uses the following logic:

For `i` in `pi_lines`, use `+=` to add the string of `i`, applying the `.strip()` method.

Check the value of `pi_string` by printing the first 300 characters using

```
pi_string[0:300]
```

- b) Once you have `pi_string`, you can find the decimal position of the birthday using the `.index()` method. You should end up with something that looks like this, where `'NNNNNN'` should be replaced by the birthday that you're testing:

```
birthday_position = pi_string.index('NNNNNN') - 1
```

Why the -1 at the end? Include a comment in the code below this variable to provide your explanation. (Consider that indexing starts at zero, and that `pi_lines` starts with 3 and a decimal point.)

11. Now you have the decimal position of a birthday in pi if it *is* found. How can you combine that into the birthday check function so that it's all part of one function call?
- a) In a new cell, copy and paste `pi_bday_check2` and rename it to `pi_bday_check3()`
- b) At the top of the function, add a new starting local variable for `bday_position` set to a blank string.
- c) Following the second `if` block (that starts with `if birthday_found != 1:`), add an `else` block that calculates `bday_position` (duplicating the calculation you previously tested out with the global variable `birthday_position`). Then add another line to the `else` block to print the following output:

```
Your birthday begins at decimal place {bday_position}
```

12. Review your code as a whole to check that it is readable and that things are happening in the right order. Add code comments or markdown blocks that you think would be helpful to a person reviewing your notebook.
13. Remember to save your changes, then commit and push all changes to week-06 when you are done.

Exercise 4.B Working with exceptions

Lab 1

Create a new folder in week-06 named **Ex4B_TryExcept**. The following exercises should be placed there.

1. In VS Code, make a new file called **exception_basics.py**.
2. For each of the following exceptions, create a short code block that raises that exception. Then put it within a try-except block to handle the error, an else block to print the result if no error occurs, and a finally statement like “Let’s try another one”:

ValueError
NameError
TypeError
SyntaxError

You may need to do a little research to come up with ideas for how to generate each exception. You will also likely notice that for most exceptions, there’s a range of different specific error that might raise that exception. If you find two different ways of producing an exception, keep both in your script for reference.

example:

```
try:
    m = banana
except NameError:
    print("NameError: Oops, looks like you tried to assign an
        undefined object to a variable")
else:
    print(m)
finally:
    print("Let's try another one...")
```

3. The above list of exceptions is not exhaustive. Take a little time to review some additional examples listed by w3schools at the following page:

https://www.w3schools.com/python/python_ref_exceptions.asp

Lab 2 – OPTIONAL (prerequisite Ex 2.D Lab 2)

1. Find the script you previously created in Functions called `name_game_bonus.py`. Make a copy to the TryExcept folder, and rename it **`name_game_bonus_exceptions.py`**.
2. Try inputting different inputs that should be invalid, like integers, single letters, or no text at all (just hitting enter at the prompt).
 - a) Are there inputs that you expect to throw an error which actually work? Why does the example work? What logic might you use to check for this type of issue?
 - b) Use comments in your script to make note of any exception(s) thrown. What is the exception type, and where does it occur?
3. Pull up your favorite generative AI tool (Claude, ChatGPT, Gemini, Copilot, or other). Start with the following prompt:

I am currently a student in an Introduction to Data Analytics bootcamp, learning Python. Please help me identify how I can add error handling to my code with try/except. I will provide a starting script, and then specific examples of errors to work through.

Copy your `name_game_bonus_exceptions.py` script into the chat. Then one by one, work through the exceptions that you identified as being thrown by different inputs, and ask for suggested try/except logic to incorporate into your script. Work iteratively, incorporating one change at a time, and including explanatory comments to help keep track of what each part of the code is doing. When you are finished, make sure to copy the resulting suggestions back into your script in week-06/Ex4B_TryExcept.

4. **CHALLENGE:** In the context of exception handling, what does the statement `raise SystemExit(0)` mean? When might it be used instead of `break`? When might it produce unexpected results? Add a comment to the bottom of your script with your answer.
5. Remember to save, commit, and sync all changes when you are done!

Module 5

NumPy

Exercise 5.A NumPy fundamentals

1. In DataAnalytics/week-06, use either Jupyter Notebook or VS Code to create a new notebook named **Ex5A_NumPy_fundamentals.ipynb**, then fill in the notebook with well-formatted markdown and code blocks to address the questions below.
2. As ever, be sure to include a brief description of the exercise in a markdown cell at the top.
3. Start by importing `numpy` as `np` in the first code cell of the notebook.

- a) If you have not yet installed the `numpy` library, you will need to do so! Open Git Bash and in your home directory enter the command:

```
pip install numpy
```

If you already have the NumPy library installed, you'll get the message "Requirement already satisfied." Otherwise, the installation should proceed and should not take longer than a couple minutes. If you encounter problems, reach out to your tech support contact.

4. Add a markdown cell with the header **## Array Basics**.
5. Add a new code cell **for each** of the following items:
 - a) Create a standard Python list with the variable name **fruits**, containing the values *apple*, *banana*, *cherry*, and *date*
 - b) Create a variable named **arr_1** as a one-dimensional array containing `fruits` then print `arr_1`.
 - c) Write an expression to check the data type of values contained in `arr_1`. After you run the cell, add a comment to explain what data type is returned. For the `dtype`, what does the letter mean, and what does the number mean?
 - d) Create a variable named **arr_2** as a two-dimensional array containing *(10, 11, 12)* and *(13, 14, 15)* then print `arr_2`.

- e) Use the transpose attribute to print `arr_2` with its dimensions transposed (`arr_2.T`). In another cell, check: does this actually alter the value stored in `arr_2`? Include your answer in a comment.
 - f) Create a variable named **`arr_3`** as a two-dimensional array containing (1, 3, 5) and (2, 4, 6) then print `arr_3`.
 - g) Create a variable named **`arr_4`** as a one-dimensional array containing two values:
using slicing notation, the element of `arr_3` at index position 0, 2 and the element of `arr_3` at index position 1, 1. Then print `arr_4`.
 - h) Create a variable named **`arr_5`** as a three-dimensional array containing all 1's, with two values in each dimension. Then print `arr_5`.
6. Close and shut down the notebook when you are finished. As always, remember to commit and push your changes to the week-06 folder!

End of Week 6 Lab Workbook